GROUP BY Problems

1.

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Courses`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| student     | varchar |
| class       | varchar |
+-------------+---------+
(student, class) is the primary key
(combination of columns with unique values)
for this table.
Each row of this table indicates the name of
a student and the class in which they are
enrolled.
```

Write a solution to find all the classes that have **at least five students**.

Return the result table in **any order**.

The result format is in the following example.

MySQL | Auto

```
# Write your MySQL query statement below
SELECT class
FROM Courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

Saved

Testcase | Test Result

Accepted Runtime: 74 ms

Case 1

Input

Courses =

```
| student | class    |
| ------- | -------- |
| A       | Math     |
| B       | English  |
| C       | Math     |
| D       | Biology  |
| E       | Math     |
| F       | Computer |
```

View more

2.

Description | Editorial | Solutions | Submissions

619. Biggest Single Number

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `MyNumbers`

```
+-------------+------+
| Column Name | Type |
+-------------+------+
| num         | int  |
+-------------+------+
This table may contain duplicates (In other words, there is no primary key for
this table in SQL).
Each row of this table contains an integer.
```

A **single number** is a number that appeared only once in the `MyNumbers` table.

Find the largest **single number**. If there is no **single number**, report `null`.

The result format is in the following example.

Example 1:

```
Input:
MyNumbers table:
```

956 | 128 | 12 Online

Code

MySQL | Auto

```
# Write your MySQL query statement
SELECT MAX(num) AS num
FROM (
    SELECT num
    FROM MyNumbers
    GROUP BY num
    HAVING COUNT(num) = 1
) AS t;
```

Saved | Ln 9, Col 1

Testcase | Test Result

Accepted Runtime: 81 ms

Case 1 | Case 2

Input

MyNumbers =

```
| num |
| --- |
| 8   |
| 8   |
| 3   |
| 3   |
| 1   |
| 4   |
```

3.

Description | Editorial | Solutions | Submissions

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Activity`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| player_id     | int     |
| device_id     | int     |
| event_date    | date    |
| games_played  | int     |
+---------------+---------+
(player_id, event_date) is the primary key (combination of columns with unique values) of this table.
This table shows the activity of players of some games.
Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.
```

Write a solution to find the **first login date** for each player.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

1.1K | 91 | 19 Online

Code

MySQL | Auto

```
# Write your MySQL query statement below
SELECT player_id, MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

Saved | Ln 5, Col 1

Testcase | Test Result

Accepted Runtime: 117 ms

Case 1

Input

Activity =

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output

player_id	first_login
1	2016-03-01

4.

1693. Daily Leads and Partners

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `DailySales`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| date_id     | date    |
| make_name   | varchar |
| lead_id     | int     |
| partner_id  | int     |
+-------------+---------+
There is no primary key (column with unique values) for this table. It may contain duplicates.
This table contains the date and the name of the product sold and the IDs of the lead and partner it was sold to.
The name consists of only lowercase English letters.
```

For each `date_id` and `make_name`, find the number of **distinct** `lead_id`'s and **distinct** `partner_id`'s.

Return the result table in **any order**.

The result format is in the following example.

MySQL | Auto

```
    COUNT(DISTINCT partner_id) AS unique_partners
FROM DailySales
GROUP BY date_id, make_name;
```

Saved

Testcase | Test Result

Accepted Runtime: 62 ms

Case 1

Input

DailySales =

date_id	make_name	lead_id	partner_id
2020-12-8	toyota	0	1
2020-12-8	toyota	1	0
2020-12-8	toyota	1	2
2020-12-7	toyota	0	2
2020-12-7	toyota	0	1
2020-12-8	honda	1	2

View more

5.

Description | Editorial | Solutions | Submissions

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Employees`

```
+-------------+------+
| Column Name | Type |
+-------------+------+
| emp_id      | int  |
| event_day   | date |
| in_time     | int  |
| out_time    | int  |
+-------------+------+
(emp_id, event_day, in_time) is the primary key (combinations of columns with unique values) of this table.
The table shows the employees' entries and exits in an office.
event_day is the day at which this event happened, in_time is the minute at which the employee entered the office, and out_time is the minute at which they left the office.
in_time and out_time are between 1 and 1440.
It is guaranteed that no two events on the same day intersect in time, and in_time < out_time.
```

Write a solution to calculate the total time **in minutes** spent by each employee on each day at the office. Note that within one day, an employee can enter and leave more than once. The time spent in the office for a single entry is `out_time - in_time`.

Return the result table in **any order**.

803 | 47 | 9 Online

Code

MySQL | Auto

```
# Write your MySQL query statement below
SELECT

    event_day as day,
    emp_id,
    SUM(out_time - in_time) AS total_time
FROM Employees
GROUP BY emp_id, event_day;
```

Saved

Testcase | Test Result

Employees =

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-3	1	42
2	2020-11-28	3	33
2	2020-12-9	47	74

Output

day	emp_id	total_time
2020-11-28	1	173
2020-12-03	1	41
2020-11-28	2	30
2020-12-09	2	27

6.

Description | Editorial | Solutions | Submissions

1050. Actors and Directors who Cooperated At Least Three Times

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `ActorDirector`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| actor_id    | int     |
| director_id | int     |
| timestamp   | int     |
+-------------+---------+
timestamp is the primary key (column with unique values) for this table.
```

Write a solution to find all the pairs `(actor_id, director_id)` where the actor has cooperated with the director at least three times.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

782 | 53 | 12 Online

Code

MySQL | Auto

```
# Write your MySQL query statement below
SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING COUNT(*) >= 3;
```

Saved

Testcase | Test Result

Input

ActorDirector =

actor_id	director_id	timestamp
1	1	0
1	1	1
1	1	2
1	2	3
1	2	4
2	1	5

View more

Output

actor_id	director_id
1	1

7.

1890. The Latest Login in 2020

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Logins`

```
+----------------+----------+
| Column Name    | Type     |
+----------------+----------+
| user_id        | int      |
| time_stamp     | datetime |
+----------------+----------+
(user_id, time_stamp) is the primary key
(combination of columns with unique values)
for this table.
Each row contains information about the
login time for the user with ID user_id.
```

Write a solution to report the **latest** login for all users in the year `2020`. Do **not** include the users who did not login in `2020`.

Return the result table **in any order**.

The result format is in the following example.

MySQL | Auto

```
SELECT user_id, MAX(time_stamp) AS last_stamp
FROM Logins
WHERE YEAR(time_stamp) = 2020
GROUP BY user_id;
```

Saved Ln

Testcase | Test Result

Accepted Runtime: 68 ms

Case 1

Input

Logins =

user_id	time_stamp
6	2020-06-30 15:06:07
6	2021-04-21 14:06:06
6	2019-03-07 00:18:15
8	2020-02-01 05:10:53
8	2020-12-30 00:46:50
2	2020-01-16 02:49:50

View more

Output

8.

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Queries`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| query_name    | varchar |
| result        | varchar |
| position      | int     |
| rating        | int     |
+---------------+---------+
This table may have duplicate rows.
This table contains information collected
from some queries on a database.
The position column has a value from 1 to
500.
The rating column has a value from 1 to 5.
Query with rating less than 3 is a poor
query.
```

We define query `quality` as:

> The average of the ratio between query rating and its position.

We also define `poor query percentage` as:

MySQL | Auto

```mysql
SELECT
    query_name,
    ROUND(AVG(rating / position), 2) AS quality,
    ROUND(100 * SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END) / COUNT(*), 2)
        AS poor_query_percentage
FROM Queries
GROUP BY query_name;
```

Saved Ln 8, Col 2

Testcase | Test Result

Accepted Runtime: 69 ms

Case 1

Input

Queries =

query_name	result	position	rating
Dog	Golden Retriever	1	5
Dog	German Shepherd	2	5
Dog	Mule	200	1
Cat	Shirazi	5	2
Cat	Siamese	3	3
Cat	Sphynx	7	4

9.

1141. User Activity for the Past 30 Days I

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Activity`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| user_id       | int     |
| session_id    | int     |
| activity_date | date    |
| activity_type | enum    |
+---------------+---------+
This table may have duplicate rows.
The activity_type column is an ENUM (category) of type ('open_session',
'end_session', 'scroll_down', 'send_message').
The table shows the user activities for a social media website.
Note that each session belongs to exactly one user.
```

Write a solution to find the daily active user count for a period of `30` days ending `2019-07-27` inclusively. A user was active on someday if they made at least one activity on that day.

Return the result table in **any order**.

The result format is in the following example.

```mysql
# Write your MySQL query statement below
SELECT
    activity_date AS day,
    COUNT(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN '2019-06-28' AND '2019-07-27'
GROUP BY activity_date;
```

Saved

Testcase | Test Result

Accepted Runtime: 72 ms

Case 1

Input

Activity =

user_id	session_id	activity_date	activity_type
1	1	2019-07-20	open_session
1	1	2019-07-20	scroll_down
1	1	2019-07-20	end_session
2	4	2019-07-20	open_session
2	4	2019-07-21	send_message
2	4	2019-07-21	end_session

View more

10.

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Courses`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| student     | varchar |
| class       | varchar |
+-------------+---------+
(student, class) is the primary key
(combination of columns with unique values)
for this table.
Each row of this table indicates the name of
a student and the class in which they are
enrolled.
```

Write a solution to find all the classes that have **at least five students**.

Return the result table in **any order**.

The result format is in the following example.

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT class
FROM Courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

Saved

Testcase | Test Result

Accepted Runtime: 74 ms

Case 1

Input

Courses =

```
| student | class    |
| ------- | -------- |
| A       | Math     |
| B       | English  |
| C       | Math     |
| D       | Biology  |
| E       | Math     |
| F       | Computer |
```

View more

Joins problems.

1.

1378. Replace Employee ID With The Unique Identifier

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Employees`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| id            | int     |
| name          | varchar |
+---------------+---------+
id is the primary key (column with unique
values) for this table.
Each row of this table contains the id and
the name of an employee in a company.
```

Table: `EmployeeUNI`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| id            | int     |
| unique_id     | int     |
+---------------+---------+
```

2K | 216 | 45 Online

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT e.name, eu.unique_id
FROM Employees e
LEFT JOIN EmployeeUNI eu
ON e.id = eu.id;
```

Saved

Testcase | Test Result

Accepted Runtime: 86 ms

Case 1

Input

Employees =

```
| id | name     |
| -- | -------- |
| 1  | Alice    |
| 7  | Bob      |
| 11 | Meir     |
| 90 | Winston  |
| 3  | Jonathan |
```

EmployeeUNI =

```
| id | unique_id |
| -- | --------- |
| 3  | 1         |
| 11 | 2         |
```

2.

1068. Product Sales Analysis I

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Sales`

```
+-------------+-------+
| Column Name | Type  |
+-------------+-------+
| sale_id     | int   |
| product_id  | int   |
| year        | int   |
| quantity    | int   |
| price       | int   |
+-------------+-------+
(sale_id, year) is the primary key
(combination of columns with unique values)
of this table.
product_id is a foreign key (reference
column) to Product table.
Each row of this table shows a sale on the
product product_id in a certain year.
Note that the price is per unit.
```

Table: `Product`

1.5K 173 30 Online

MySQL Auto

```
SELECT p.product_name, s.year, s.price
FROM Sales s
JOIN Product p
ON s.product_id = p.product_id;
```

Saved Ln 5,

Testcase | Test Result

Accepted Runtime: 115 ms

Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product =

product_id	product_name
100	Nokia
200	Apple
300	Samsung

3.

Table: `Bonus`

```
+-------------+-------+
| Column Name | Type  |
+-------------+-------+
| empId       | int   |
| bonus       | int   |
+-------------+-------+
empId is the column of unique values for this
table.
empId is a foreign key (reference column) to
empId from the Employee table.
Each row of this table contains the id of an
employee and their respective bonus.
```

Write a solution to report the name and bonus amount of each employee who satisfies either of the following:

- The employee has a bonus **less than** `1000`.
- The employee did not get any bonus.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

1.6K 219 49 Online

MySQL Auto

```
# Write your MySQL query statement below
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b
ON e.empId = b.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

Saved

Testcase | Test Result

Accepted Runtime: 117 ms

Case 1

Input

Employee =

empId	name	supervisor	salary
3	Brad	null	4000
1	John	3	1000
2	Dan	3	2000
4	Thomas	3	4000

Bonus =

empId	bonus
2	500

4.

1280. Students and Examinations

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Students`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| student_id    | int     |
| student_name  | varchar |
+---------------+---------+
student_id is the primary key (column with
unique values) for this table.
Each row of this table contains the ID and
the name of one student in the school.
```

Table: `Subjects`

```
+--------------+---------+
| Column Name  | Type    |
+--------------+---------+
| subject_name | varchar |
+--------------+---------+
subject_name is the primary key (column with
unique values) for this table.
```

3K 462 71 Online

MySQL Auto

```mysql
SELECT
    s.student_id,
    s.student_name,
    sub.subject_name,
    COUNT(e.subject_name) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
ON s.student_id = e.student_id
AND sub.subject_name = e.subject_name
GROUP BY s.student_id, s.student_name, sub.subject_name
ORDER BY s.student_id, sub.subject_name;
```

Saved Ln 13, C

Testcase | Test Result

Accepted Runtime: 139 ms

Case 1

Input

Students =

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

5.

570. Managers with at Least 5 Direct Reports

Medium | Topics | Companies | Hint

SQL Schema > Pandas Schema >

Table: `Employee`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| name        | varchar |
| department  | varchar |
| managerId   | int     |
+-------------+---------+
id is the primary key (column with unique
values) for this table.
Each row of this table indicates the name of
an employee, their department, and the id of
their manager.
If managerId is null, then the employee does
not have a manager.
No employee will be the manager of themself.
```

MySQL Auto

```mysql
# Write your MySQL query statement below
SELECT name
FROM Employee
WHERE id IN (
    SELECT managerId
    FROM Employee
    GROUP BY managerId
    HAVING COUNT(*) >= 5
);
```

Saved

Testcase | Test Result

Accepted Runtime: 108 ms

Case 1

Input

Employee =

id	name	department	managerId

6.

```mysql
SELECT SUM(CITY.Population)
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Asia';
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
27028484
```

7.

```sql
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Asia 693038
Europe 175138
Oceania 109189
South America 147435
Africa 274439
```

8.

Write a solution to find the `person_name` of the **last person** that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

Note that *only one* person can board the bus at any given turn.

The result format is in the following example.

Example 1:

Input:

Queue table:

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Output:

person_name
John Cena

1.1K 163 22 Online

MySQL Auto

```sql
# Write your MySQL query statement below
SELECT person_name
FROM (
    SELECT person_name,
           SUM(weight) OVER (ORDER BY turn) AS total_weight
    FROM Queue
) t
WHERE total_weight <= 1000
ORDER BY total_weight DESC
LIMIT 1;
```

Saved Ln 10, C

Testcase | Test Result

Accepted Runtime: 103 ms

Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3

9.

1164. Product Price at a Given Date

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Products`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| product_id    | int     |
| new_price     | int     |
| change_date   | date    |
+---------------+---------+
(product_id, change_date) is the primary key
(combination of columns with unique values)
of this table.
Each row of this table indicates that the
price of some product was changed to a new
price at some date.
```

Initially, all products have price 10.

Write a solution to find the prices of all products on the date `2019-08-16`.

Return the result table in **any order**.

The result format is in the following example.

1.4K | 216 | 35 Online

MySQL | Auto

```mysql
SELECT p.product_id,
       COALESCE(pr.new_price, 10) AS price
FROM (SELECT DISTINCT product_id FROM Products) p
LEFT JOIN (
    SELECT product_id, new_price
    FROM Products
    WHERE (product_id, change_date) IN (
        SELECT product_id, MAX(change_date)
        FROM Products
        WHERE change_date <= '2019-08-16'
        GROUP BY product_id
    )
) pr
ON p.product_id = pr.product_id;
```

Saved | Ln 13, Col

Testcase | Test Result

Accepted Runtime: 75 ms

Case 1

Input

Products =

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14

Subquery Problems

1.

Description | Editorial | Solutions | Subm

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Employee`

```
+--------------+---------+
| Column Name  | Type    |
+--------------+---------+
| id           | int     |
| name         | varchar |
| salary       | int     |
| managerId    | int     |
+--------------+---------+
id is the primary key (column with
unique values) for this table.
Each row of this table indicates the ID
of an employee, their name, salary, and
the ID of their manager.
```

Write a solution to find the employees who earn more than their managers.

Return the result table in **any order**.

The result format is in the following example.

3.1K | 148 | 56 Onl

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m
ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Saved | Ln 6, Col

Testcase | Test Result

Employee =

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	null
4	Max	90000	null

Output

Employee
Joe

Expected

Employee
Joe

2.

183. Customers Who Never Order

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Customers`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| name        | varchar |
+-------------+---------+
id is the primary key (column with
unique values) for this table.
Each row of this table indicates the ID
and name of a customer.
```

Table: `Orders`

```
+-------------+------+
| Column Name | Type |
+-------------+------+
| id          | int  |
| customerId  | int  |
```

3K | 161 | 37 Onl

MySQL | Auto

```
# Write your MySQL query statement below
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o
ON c.id = o.customerId
WHERE o.id IS NULL;
```

Saved | Ln 6, Col 20

Testcase | Test Result

Accepted Runtime: 102 ms

Case 1

Input

Customers =

id	name
1	Joe
2	Henry
3	Sam
4	Max

Orders =

id	customerId
1	3

3.

197. Rising Temperature

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Weather`

```
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| id            | int     |
| recordDate    | date    |
| temperature   | int     |
+---------------+---------+
id is the column with unique values for
this table.
There are no different rows with the
same recordDate.
This table contains information about
the temperature on a certain day.
```

Write a solution to find all dates' `id` with higher temperatures compared to its previous dates (yesterday).

Return the result table in **any order**.

The result format is in the following example.

4.2K | 608 | 71 Online

MySQL | Auto

```
# Write your MySQL query statement below
SELECT w1.id
FROM Weather w1
JOIN Weather w2
ON w1.recordDate = DATE_ADD(w2.recordDate, INTERVAL 1
DAY)
WHERE w1.temperature > w2.temperature;
```

Saved | Ln 6, Col

Testcase | Test Result

Accepted Runtime: 74 ms

Case 1

Input

Weather =

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

Output

id
2
4

4.

607. Sales Person

Easy | Topics | Companies | Hint

SQL Schema > Pandas Schema >

Table: `SalesPerson`

```
+-----------------+---------+
| Column Name     | Type    |
+-----------------+---------+
| sales_id        | int     |
| name            | varchar |
| salary          | int     |
| commission_rate | int     |
| hire_date       | date    |
+-----------------+---------+
sales_id is the primary key (column
with unique values) for this table.
Each row of this table indicates the
name and the ID of a salesperson
alongside their salary, commission
rate, and hire date.
```

Table: `Company`

```
+-------------+---------+
| Column Name | Type    |
```

1.4K | 147 | 16 Online

MySQL | Auto

```
# Write your MySQL query statement below
SELECT name
FROM SalesPerson
WHERE sales_id NOT IN (
    SELECT o.sales_id
    FROM Orders o
    JOIN Company c
    ON o.com_id = c.com_id
    WHERE c.name = 'RED'
);
```

Saved | Ln 10, Co

Testcase | Test Result

Accepted Runtime: 131 ms

Case 1

Input

SalesPerson =

```
| sales_id | name | salary | commission_rate
| hire_date  |
| -------- | ---- | ------ | ---------------
| ---------- |
| 1        | John | 100000 | 6
| 4/1/2006   |
| 2        | Amy  | 12000  | 5
| 5/1/2010   |
```

5.

180. Consecutive Numbers

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Logs`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| num         | varchar |
+-------------+---------+
In SQL, id is the primary key for this table.
id is an autoincrement column starting from
1.
```

Find all numbers that appear at least three times consecutively.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

```
Input:
```

2.7K | 260 | 57 Online

```
# Write your MySQL query statement below
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM Logs l1
JOIN Logs l2 ON l1.id = l2.id - 1
JOIN Logs l3 ON l1.id = l3.id - 2
WHERE l1.num = l2.num
AND l2.num = l3.num;
```

Saved

Testcase | Test Result

Accepted Runtime: 79 ms

Case 1

Input

Logs =

```
| id | num |
| -- | --- |
| 1  | 1   |
| 2  | 1   |
```

6.

175. Combine Two Tables

Easy | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Person`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| personId    | int     |
| lastName    | varchar |
| firstName   | varchar |
+-------------+---------+
personId is the primary key (column with
unique values) for this table.
This table contains information about
the ID of some persons and their first
and last names.
```

Table: `Address`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| addressId   | int     |
| personId    | int     |
```

4.3K 167 106 Online

MySQL Auto

```mysql
# Write your MySQL query statement below
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a
ON p.personId = a.personId;
```

Saved Ln 5, Col

Testcase | Test Result

Accepted Runtime: 130 ms

Case 1

Input

Person =

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Address =

addressId	personId	city	state
1	2	New York City	New Yor

7.

178. Rank Scores

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Scores`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| score       | decimal |
+-------------+---------+
id is the primary key (column with unique values) for this
table.
Each row of this table contains the score of a game. Score is a
floating point value with two decimal places.
```

Write a solution to find the rank of the scores. The ranking should be calculated according to the following rules:

- The scores should be ranked from the highest to the lowest.
- If there is a tie between two scores, both should have the same ranking.
- After a tie, the next ranking number should be the next consecutive integer value. In other words, there should be no holes between ranks.

2.5K 153 30 Online

MySQL Auto

```mysql
# Write your MySQL query statement below
SELECT
    s1.score,
    (
        SELECT COUNT(DISTINCT s2.score)
        FROM Scores s2
        WHERE s2.score >= s1.score
    ) AS `rank`
FROM Scores s1
ORDER BY s1.score DESC;
```

Saved

Testcase | Test Result

Accepted Runtime: 69 ms

Case 1

Input

Scores =

id	score
1	3.5
2	3.65
3	4
4	3.85
5	4
6	3.65

8.

570. Managers with at Least 5 Direct Reports

Medium | Topics | Companies | Hint

SQL Schema > Pandas Schema >

Table: `Employee`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| name        | varchar |
| department  | varchar |
| managerId   | int     |
+-------------+---------+
id is the primary key (column with unique values) for this table.
Each row of this table indicates the name of an employee, their department, and the id of their manager.
If managerId is null, then the employee does not have a manager.
No employee will be the manager of themself.
```

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT name
FROM Employee
WHERE id IN (
    SELECT managerId
    FROM Employee
    GROUP BY managerId
    HAVING COUNT(*) >= 5
);
```

Saved

Testcase | Test Result

Accepted Runtime: 108 ms

Case 1

Input

Employee =

id	name	department	managerId

9.

Description | Editorial | Solutions | Submissions

Write a solution to **delete** all duplicate emails, keeping only one unique email with the smallest `id`.

For SQL users, please note that you are supposed to write a `DELETE` statement and not a `SELECT` one.

For Pandas users, please note that you are supposed to modify `Person` in place.

After running your script, the answer shown is the `Person` table. The driver will first compile and run your piece of code and then show the `Person` table. The final order of the `Person` table **does not matter**.

The result format is in the following example.

Example 1:

```
Input:
Person table:
+----+------------------+
| id | email            |
+----+------------------+
| 1  | john@example.com |
| 2  | bob@example.com  |
| 3  | john@example.com |
+----+------------------+
```

2.1K | 322 | 47 Online

Code

MySQL | Auto

```mysql
# Write your MySQL query statement below
DELETE p1
FROM Person p1
JOIN Person p2
ON p1.email = p2.email
AND p1.id > p2.id;
```

Saved

Testcase | Test Result

Accepted Runtime: 130 ms

Case 1

Input

Person =

id	email
1	john@example.com
2	bob@example.com

10.

1204. Last Person to Fit in the Bus

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Queue`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| person_id   | int     |
| person_name | varchar |
| weight      | int     |
| turn        | int     |
+-------------+---------+
```

person_id column contains unique values.
This table has the information about all people waiting for a bus.
The person_id and turn columns will contain all numbers from 1 to n, where n is the number of rows in the table.
turn determines the order of which the people will board the bus, where turn=1 denotes the first person to board and turn=n denotes the last person to board.
weight is the weight of the person in kilograms.

1.1K | 163 | 23 Online

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT person_name
FROM (
    SELECT person_name,
            SUM(weight) OVER (ORDER BY turn) AS total_weight
    FROM Queue
) t
WHERE total_weight <= 1000
ORDER BY total_weight DESC
```

Saved | Ln 1, C

Testcase | Test Result

Accepted Runtime: 84 ms

Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

11.

176. Second Highest Salary

Medium | Topics | Companies

SQL Schema > Pandas Schema >

Table: `Employee`

```
+-------------+------+
| Column Name | Type |
+-------------+------+
| id          | int  |
| salary      | int  |
+-------------+------+
```

id is the primary key (column with unique values) for this table.
Each row of this table contains information about the salary of an employee.

Write a solution to find the second highest **distinct** salary from the `Employee` table. If there is no second highest salary, return `null` `(return None in Pandas)`.

The result format is in the following example.

Example 1:

4.2K | 403 | 81 Online

MySQL | Auto

```mysql
# Write your MySQL query statement below
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Saved | Ln 1

Testcase | Test Result

Accepted Runtime: 88 ms

Case 1 | Case 2

Input

Employee =

id	salary
1	100
2	200
3	300

Output

SecondHighestSalary
200

Expected